



Training Guide — Low-Level File Operations

POSIX File I/O in C and C++

open · close · read · write · lseek · tell · stat · dup · fcntl · mmap

LOW-LEVEL FILE OPERATIONS — COMPLETE GUIDE

System Calls vs Standard Library

open / close / read / write / lseek

stat / fstat / access / truncate

dup / dup2 / fcntl / ioctl

Directory operations: opendir / readdir / closedir

Memory-mapped files: mmap / munmap

File locking: flock / fcntl locking

Platform: POSIX (Linux / macOS / Unix). All system calls shown are available on any POSIX-compliant system. Windows equivalents are noted where relevant.



Table of Contents

- [Part 1: Concepts — Two Layers of File I/O](#)
 - [Part 2: open and close](#)
 - [Part 3: read and write](#)
 - [Part 4: lseek and File Position](#)
 - [Part 5: stat, fstat, access — File Metadata](#)
 - [Part 6: truncate, ftruncate](#)
 - [Part 7: dup, dup2 — File Descriptor Duplication](#)
 - [Part 8: fcntl — File Control](#)
 - [Part 9: Directory Operations](#)
 - [Part 10: Memory-Mapped Files — mmap](#)
 - [Part 11: File Locking](#)
 - [Part 12: Complete Reference and Comparison Table](#)
-

Part 1: Concepts — Two Layers of File I/O



Learning Objectives

- Understand the difference between system calls and the C/C++ standard library
 - Know what a file descriptor is and how it differs from a FILE* stream
 - Understand buffered vs unbuffered I/O and when each matters
 - Know the three standard file descriptors (stdin, stdout, stderr)
-

The Two Layers

TWO LAYERS OF FILE I/O

HIGH-LEVEL (C Standard Library / C++ iostream)

`FILE* fopen() fclose() fread() fwrite()`
`fseek() ftell() fgets() fputs() fprintf() fscanf()`
`std::fstream ifstream ofstream`

- BUFFERED: data accumulated in user-space buffer
- Portable across all platforms (Windows, Linux, macOS)
- Handles text mode, locale, line endings automatically
- Best for: text files, formatted data, portable code

| built on top of ▼

LOW-LEVEL (POSIX System Calls)

`int open() close() read() write()`
`off_t lseek() stat() fstat() dup() fcntl()`

- UNBUFFERED: every call goes directly to the kernel
- POSIX standard (Linux, macOS, Unix – NOT native Windows)
- Access to file metadata, permissions, locking, mmap
- Best for: binary files, performance, IPC, network sockets

What Is a File Descriptor?

FILE DESCRIPTOR (fd):

A small NON-NEGATIVE INTEGER returned by `open()`. It is the kernel's handle to an open file – similar to `FILE*` but lower level.

Every process has a FILE DESCRIPTOR TABLE:

fd	Points to
0	stdin (standard input – keyboard)
1	stdout (standard output – terminal)
2	stderr (standard error – terminal)
3	first file you open → returned by <code>open()</code>
4	second file opened
...	...

Limits:

- Default per-process limit: 1024 file descriptors (`ulimit -n`)
- Kernel-wide limit: `/proc/sys/fs/file-max`
- Can be raised with: `ulimit -n 65536`

fd vs `FILE*`:

	File Descriptor	<code>FILE*</code> stream
Type	int	<code>FILE*</code> (opaque struct)
Buffering	None (kernel)	User-space buffer

Layer	System call	Standard library
Portability	POSIX only	C89 – everywhere
Conversion	fileno(fp) → fd	fdopen(fd) → FILE*

Headers Required

```
// POSIX system call headers
#include <fcntl.h>      // open(), O_RDONLY, O_WRONLY, O_CREAT, etc.
#include <unistd.h>     // read(), write(), close(), lseek(), dup(), etc.
#include <sys/stat.h>   // stat(), fstat(), chmod(), mode flags S_IRUSR, etc.
#include <sys/types.h>  // off_t, ssize_t, pid_t, etc.
#include <dirent.h>     // opendir(), readdir(), closedir()
#include <sys/mman.h>    // mmap(), munmap()
#include <sys/file.h>    // flock()
#include <errno.h>       // errno, ENOENT, EACCES, etc.
#include <string.h>      // strerror()
#include <iostream>     // for C++ output
```

Part 2: open and close

The open() System Call

`open()` is the gateway to low-level file I/O. It opens a file and returns a file descriptor.

SIGNATURE:

```
#include <fcntl.h>
```

```
int open(const char* pathname, int flags);  
int open(const char* pathname, int flags, mode_t mode);
```

Returns: file descriptor (≥ 0) on success
-1 on error (and sets `errno`)

`pathname` → path to the file (absolute or relative)
`flags` → how to open (read, write, create, etc.) – OR'd together
`mode` → permissions for NEW files (only used with `O_CREAT`)

Opening Flags — The Complete List

open() FLAGS

ACCESS MODE (exactly one of these MUST be specified):

O_RDONLY Open for reading only
O_WRONLY Open for writing only
O_RDWR Open for reading and writing

CREATION FLAGS (optional, OR'd in):

O_CREAT Create file if it doesn't exist (needs mode)
O_EXCL Fail if file already exists (used with O_CREAT)
O_TRUNC Truncate file to 0 bytes if it exists
O_APPEND Always write to end of file (atomic)

STATUS FLAGS (optional, OR'd in):

O_NONBLOCK Open in non-blocking mode (won't block on read)
O_SYNC Synchronous I/O (writes go to disk before return)
O_DSYNC Sync data only (not metadata) before return
O_CLOEXEC Close fd automatically on exec()
O_NOFOLLOW Don't follow symbolic links
O_DIRECTORY Fail if path is not a directory

File Permission Mode Bits

MODE BITS (for O_CREAT – the permission of the NEW file):

S_IRUSR	(0400)	Owner can read
S_IWUSR	(0200)	Owner can write
S_IXUSR	(0100)	Owner can execute
S_IRGRP	(0040)	Group can read
S_IWGRP	(0020)	Group can write
S_IXGRP	(0010)	Group can execute
S_IROTH	(0004)	Others can read
S_IWOTH	(0002)	Others can write
S_IXOTH	(0001)	Others can execute

Common combinations:

0644 = rw-r--r-- (owner rw, group r, others r) ← typical data file
0755 = rwxr-xr-x (owner rwx, group rx, others rx) ← executable
0600 = rw----- (owner rw only) ← private data
0777 = rwxrwxrwx (everyone full) ← avoid! security risk

IMPORTANT: actual permissions = mode & ~umask

If umask is 022, creating with 0666 gives 0644 (022 bits removed)

The close() System Call

SIGNATURE:

```
int close(int fd);
```

Returns: 0 on success, -1 on error

ALWAYS close file descriptors when done:

- Releases kernel resources
- Flushes kernel buffers for O_SYNC files
- Decrements reference count on the open file description
- Allows other processes to lock the file
- File descriptor becomes available for reuse

COMMON MISTAKE: ignoring the return value of close()

On NFS (network file systems), close() can fail (write error).

Always check: `if (close(fd) == -1) { perror("close"); }`

Complete open/close Program

```
// file: open_close_demo.cpp
// Demonstrates open(), close(), and all major flag combinations

#include <fcntl.h>
#include <unistd.h>
#include <sys/stat.h>
#include <iostream>
#include <cstring>
#include <cerrno>

// Helper: print errno message
static void die(const std::string& msg) {
    std::cerr << "ERROR [" << msg << "]: " << strerror(errno) << "\n";
}

int main() {
    std::cout << "
    open() and close() - Complete Demo
    \n\n";

    // — 1. Create a new file (O_CREAT | O_WRONLY) —————
    std::cout << "— 1. Create new file —\n";
    int fd = open("demo.txt",
                  O_WRONLY | O_CREAT | O_TRUNC,
                  S_IRUSR | S_IWUSR | S_IRGRP | S_IROTH); // 0644

    if (fd == -1) {
        die("open for create");
    }
}
```

```
    return 1;
}
std::cout << "Created demo.txt | fd=" << fd << "\n";
const char* content = "Hello, Low-Level File I/O!\nLine 2 of data.\n";
write(fd, content, strlen(content));

if (close(fd) == -1) {
    die("close after create");
    return 1;
}
std::cout << "Closed fd=" << fd << "\n\n";

// — 2. Open for reading only (O_RDONLY) —————
std::cout << "— 2. Open for reading —\n";
fd = open("demo.txt", O_RDONLY);
if (fd == -1) { die("open O_RDONLY"); return 1; }
std::cout << "Opened for read | fd=" << fd << "\n";
close(fd);

// — 3. Open for read/write (O_RDWR) —————
std::cout << "\n— 3. Open for read/write —\n";
fd = open("demo.txt", O_RDWR);
if (fd == -1) { die("open O_RDWR"); return 1; }
std::cout << "Opened for read/write | fd=" << fd << "\n";
close(fd);

// — 4. Append mode (O_APPEND) —————
std::cout << "\n— 4. Append mode —\n";
fd = open("demo.txt", O_WRONLY | O_APPEND);
if (fd == -1) { die("open O_APPEND"); return 1; }
std::cout << "Opened for append | fd=" << fd << "\n";
const char* extra = "Appended line.\n";
```

```
write(fd, extra, strlen(extra));
close(fd);

// — 5. Exclusive creation (O_EXCL) — fails if file exists —————
std::cout << "\n— 5. Exclusive create (O_EXCL) —\n";
fd = open("demo.txt", O_CREAT | O_EXCL | O_WRONLY, 0644);
if (fd == -1) {
    // Expected: EEXIST
    std::cout << "Expected failure: " << strerror(errno)
              << " (errno=" << errno << ")\n";
}

// O_EXCL succeeds on a NEW file:
fd = open("new_exclusive.txt", O_CREAT | O_EXCL | O_WRONLY, 0644);
if (fd != -1) {
    std::cout << "O_EXCL succeeded on new file | fd=" << fd << "\n";
    close(fd);
    unlink("new_exclusive.txt"); // cleanup
}

// — 6. O_CLOEXEC: close on exec —————
std::cout << "\n— 6. O_CLOEXEC —\n";
fd = open("demo.txt", O_RDONLY | O_CLOEXEC);
if (fd != -1) {
    std::cout << "Opened with O_CLOEXEC | fd=" << fd << "\n";
    std::cout << " → fd will be automatically closed if exec() is called\n";
    close(fd);
}

// — 7. Multiple files open simultaneously —————
std::cout << "\n— 7. Multiple files open simultaneously —\n";
int fds[3];
```

```
const char* names[] = {"file_a.txt", "file_b.txt", "file_c.txt"};

for (int i = 0; i < 3; i++) {
    fds[i] = open(names[i], O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (fds[i] == -1) { die("open multi"); return 1; }
    std::cout << "Opened " << names[i] << " | fd=" << fds[i] << "\n";
}

for (int i = 0; i < 3; i++) {
    if (close(fds[i]) == 0)
        std::cout << "Closed fd=" << fds[i] << "\n";
    unlink(names[i]);
}

// — 8. errno values on failure —————
std::cout << "\n— 8. Common errno values —\n";
struct { const char* path; int flags; const char* desc; } tests[] = {
    {"/no/such/directory/file.txt", O_RDONLY, "ENOENT (no such file)"},
    {"/root/secret.txt",           O_RDONLY, "EACCES (permission denied)"},
    {nullptr,                     0,       nullptr}
};

for (int i = 0; tests[i].path; i++) {
    fd = open(tests[i].path, tests[i].flags);
    if (fd == -1)
        std::cout << tests[i].desc << ": " << strerror(errno) << "\n";
    else
        close(fd);
}

// Cleanup
unlink("demo.txt");
```

```
std::cout << "\nAll done.\n";  
return 0;  
}
```

Part 3: read and write

The read() System Call

SIGNATURE:

```
#include <unistd.h>
```

```
ssize_t read(int fd, void* buf, size_t count);
```

fd → open file descriptor

buf → buffer to read data INTO

count → maximum bytes to read

Returns:

> 0 → number of bytes actually read

= 0 → end of file reached

= -1 → error (errno set)

CRITICAL: read() may return FEWER bytes than requested!

This is normal – it is NOT an error. Always loop until you have read all the bytes you need.

Reasons for short reads:

- Reached end of file
- Signal interrupted the call (errno == EINTR) → retry
- Reading from a pipe/socket – may not have count bytes yet
- Kernel transferred partial buffer

The write() System Call

SIGNATURE:

```
ssize_t write(int fd, const void* buf, size_t count);
```

Returns:

- ≥ 0 → number of bytes actually written
- $= -1$ → error (errno set)

CRITICAL: write() may also return fewer bytes than requested!

Always check and loop until all bytes are written.

Reasons for short writes:

- Disk full (ENOSPC)
- Signal interrupted (EINTR)
- Non-blocking fd with full buffer (EAGAIN)
- Writing to a pipe with limited buffer

Complete read/write Program

```
// file: read_write_demo.cpp

#include <fcntl.h>
#include <unistd.h>
#include <sys/stat.h>
#include <iostream>
#include <cstring>
#include <cerrno>
#include <cassert>

// — Helper: write all bytes reliably (handles short writes) —————
ssize_t writeAll(int fd, const void* buf, size_t count) {
    const char* ptr = static_cast<const char*>(buf);
    size_t total = 0;

    while (total < count) {
        ssize_t n = write(fd, ptr + total, count - total);
        if (n == -1) {
            if (errno == EINTR) continue; // interrupted by signal, retry
            return -1; // real error
        }
        total += static_cast<size_t>(n);
    }
    return static_cast<ssize_t>(total);
}

// — Helper: read exactly 'count' bytes (handles short reads) —————
// Returns bytes read; 0 on EOF; -1 on error
```

```
ssize_t readAll(int fd, void* buf, size_t count) {
    char* ptr = static_cast<char*>(buf);
    size_t total = 0;

    while (total < count) {
        ssize_t n = read(fd, ptr + total, count - total);
        if (n == -1) {
            if (errno == EINTR) continue; // signal interrupted, retry
            return -1;
        }
        if (n == 0) break; // EOF
        total += static_cast<size_t>(n);
    }
    return static_cast<ssize_t>(total);
}
```

// — Helper: read entire file into string —————

```
std::string readEntireFile(int fd) {
    std::string result;
    char buf[4096];
    ssize_t n;
    while ((n = read(fd, buf, sizeof(buf))) > 0)
        result.append(buf, static_cast<size_t>(n));
    return result;
}
```

```
int main() {
    std::cout << "┌───────────────────────────────────┐\n";
    std::cout << "│   read() and write() – Complete Demo   │\n";
    std::cout << "└───────────────────────────────────┘\n\n";
}
```

// — 1. Basic write —————

```
std::cout << "— 1. Basic write —\n";
int fd = open("rw_demo.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
if (fd == -1) { perror("open"); return 1; }

const char* msg = "The quick brown fox jumps over the lazy dog.\n";
ssize_t written = write(fd, msg, strlen(msg));
std::cout << "write() returned: " << written << " bytes\n";

// Write more data
for (int i = 1; i <= 5; i++) {
    char line[64];
    int len = snprintf(line, sizeof(line), "Line %d: data value = %d\n",
                       i, i * i);
    writeAll(fd, line, static_cast<size_t>(len));
}
close(fd);

// — 2. Basic read —————
std::cout << "\n— 2. Basic read —\n";
fd = open("rw_demo.txt", O_RDONLY);
if (fd == -1) { perror("open"); return 1; }

char buf[128];
ssize_t n;
int lineNum = 1;
std::string partial;

// Read in 128-byte chunks, split into lines for display
while ((n = read(fd, buf, sizeof(buf))) > 0) {
    partial.append(buf, static_cast<size_t>(n));
}
if (n == -1) perror("read");
```

```
std::cout << "File contents:\n";
std::cout << partial;
close(fd);

// — 3. Read fixed-size records —————
std::cout << "\n— 3. Fixed-size record I/O —\n";

struct Employee {
    char    name[32];
    int     id;
    double  salary;
};

// Write records
fd = open("employees.dat", O_WRONLY | O_CREAT | O_TRUNC, 0644);
Employee employees[] = {
    {"Alice Smith", 1001, 75000.0},
    {"Bob Johnson", 1002, 82000.0},
    {"Carol Williams", 1003, 91000.0},
    {"Diana Prince", 1004, 68000.0}
};

int numEmp = sizeof(employees) / sizeof(employees[0]);

for (int i = 0; i < numEmp; i++) {
    ssize_t w = writeAll(fd, &employees[i], sizeof(Employee));
    std::cout << "Wrote record " << i+1
                << " [" << w << " bytes]: " << employees[i].name << "\n";
}
close(fd);

// Read records back
```

```

fd = open("employees.dat", O_RDONLY);
std::cout << "\nReading records:\n";
Employee emp;
int rec = 1;
while ((n = readAll(fd, &emp, sizeof(Employee))) == sizeof(Employee)) {
    std::cout << "  Record " << rec++ << ": "
              << emp.name << " | ID=" << emp.id
              << " | $" << emp.salary << "\n";
}
close(fd);

// — 4. Write to stdout (fd=1) and stderr (fd=2) —————
std::cout << "\n— 4. Write directly to stdout/stderr —\n";
const char* outMsg = "This goes to stdout (fd=1)\n";
const char* errMsg = "This goes to stderr (fd=2)\n";
write(STDOUT_FILENO, outMsg, strlen(outMsg));
write(STDERR_FILENO, errMsg, strlen(errMsg));

// — 5. Binary data: write arbitrary bytes —————
std::cout << "\n— 5. Binary data I/O —\n";
fd = open("binary.dat", O_WRONLY | O_CREAT | O_TRUNC, 0644);

// Write integers as raw bytes
int values[] = {0x48454C4C, 0x4F202121, 42, -1, 0xDEADBEEF};
writeAll(fd, values, sizeof(values));
close(fd);

fd = open("binary.dat", O_RDONLY);
int readValues[5];
n = readAll(fd, readValues, sizeof(readValues));
std::cout << "Read " << n << " bytes of binary data:\n";
for (int i = 0; i < 5; i++) {

```

```
std::cout << "  values[" << i << "] = "
          << readValues[i] << " (0x";
std::cout.setf(std::ios::hex, std::ios::basefield);
std::cout << readValues[i];
std::cout.setf(std::ios::dec, std::ios::basefield);
std::cout << ")\n";
}
close(fd);

// — 6. Error handling patterns —————
std::cout << "\n— 6. Error handling —\n";

// Write to read-only fd:
fd = open("rw_demo.txt", O_RDONLY);
ssize_t w = write(fd, "test", 4);
if (w == -1)
    std::cout << "Write to O_RDONLY: " << strerror(errno) << "\n";
close(fd);

// Read from write-only fd:
fd = open("rw_demo.txt", O_WRONLY | O_APPEND);
char rbuf[10];
n = read(fd, rbuf, sizeof(rbuf));
if (n == -1)
    std::cout << "Read from O_WRONLY: " << strerror(errno) << "\n";
close(fd);

// — 7. Read in a loop until EOF —————
std::cout << "\n— 7. Read-to-EOF pattern —\n";
fd = open("rw_demo.txt", O_RDONLY);
char chunk[16];
size_t totalBytesRead = 0;
```

```
int    chunkCount = 0;

while ((n = read(fd, chunk, sizeof(chunk))) > 0) {
    totalBytesRead += static_cast<size_t>(n);
    chunkCount++;
}
if (n == -1) perror("read");

close(fd);
std::cout << "Read " << totalBytesRead << " bytes in "
           << chunkCount << " chunks of max " << sizeof(chunk) << "\n";

// Cleanup
unlink("rw_demo.txt");
unlink("employees.dat");
unlink("binary.dat");

return 0;
}
```

Part 4: lseek and File Position

Theory — The File Offset

Every open file description has a **current file offset** — the position at which the next `read()` or `write()` will occur. It starts at 0 (beginning) and advances automatically with each read/write.

`lseek()` lets you **move this offset** to any position, enabling **random access** to any part of the file.

FILE OFFSET VISUALISATION:

File contents: [H][e][l][l][o][,][][W][o][r][l][d][!][\n]

Byte position: 0 1 2 3 4 5 6 7 8 9 10 11 12 13

After `open()`: offset = 0 (beginning)

After `read(fd, 5)`: offset = 5 (read "Hello")

After `lseek(fd, 7, SEEK_SET)`: offset = 7 (set to position 7)

After `read(fd, 5)`: offset = 12 (read "World")

After `lseek(fd, 0, SEEK_SET)`: offset = 0 (back to start)

After `lseek(fd, -3, SEEK_END)`: offset = 11 (3 bytes before end)

lseek() System Call

SIGNATURE :

```
#include <unistd.h>
```

```
off_t lseek(int fd, off_t offset, int whence);
```

fd → open file descriptor

offset → byte offset (can be negative for SEEK_END, SEEK_CUR)

whence → reference point for the offset:

SEEK_SET → from the BEGINNING of the file

lseek(fd, 0, SEEK_SET) → go to byte 0 (rewind)

lseek(fd, 10, SEEK_SET) → go to byte 10

SEEK_CUR → from CURRENT position

lseek(fd, 0, SEEK_CUR) → get current position (no move)

lseek(fd, 5, SEEK_CUR) → advance 5 bytes forward

lseek(fd, -3, SEEK_CUR) → move 3 bytes backward

SEEK_END → from the END of the file

lseek(fd, 0, SEEK_END) → go to end (just past last byte)

lseek(fd, -5, SEEK_END) → 5 bytes before end

Returns: new file offset from beginning (off_t)

-1 on error

GETTING CURRENT POSITION (equivalent to ftell()):

```
off_t pos = lseek(fd, 0, SEEK_CUR);
```

GETTING FILE SIZE:

```
off_t size = lseek(fd, 0, SEEK_END);  
lseek(fd, 0, SEEK_SET);    // rewind after!
```

Complete lseek Program

```
// file: lseek_demo.cpp

#include <fcntl.h>
#include <unistd.h>
#include <sys/stat.h>
#include <iostream>
#include <cstring>
#include <cerrno>

void showPosition(int fd, const std::string& label) {
    off_t pos = lseek(fd, 0, SEEK_CUR);
    std::cout << label << " -> offset=" << pos << "\n";
}

int main() {
    std::cout << "
    lseek() - Complete Demo
    \n\n";

    // Create a file with known content
    int fd = open("seek_demo.txt", O_RDWR | O_CREAT | O_TRUNC, 0644);
    if (fd == -1) { perror("open"); return 1; }

    const char* data = "ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789";
    //
    //
    //
    write(fd, data, strlen(data));
```

```
// — 1. Get file size using lseek(SEEK_END) —————
std::cout << "— 1. Get file size —\n";
off_t fileSize = lseek(fd, 0, SEEK_END);
std::cout << "File size: " << fileSize << " bytes\n";
showPosition(fd, "After SEEK_END(0)");

// — 2. SEEK_SET: go to absolute position —————
std::cout << "\n— 2. SEEK_SET —\n";
lseek(fd, 0, SEEK_SET);
showPosition(fd, "SEEK_SET(0) – rewind");

char c;
read(fd, &c, 1); std::cout << "Read at 0: '" << c << "'\n";

lseek(fd, 10, SEEK_SET);
showPosition(fd, "SEEK_SET(10)");
read(fd, &c, 1); std::cout << "Read at 10: '" << c << "'\n";

lseek(fd, 25, SEEK_SET);
showPosition(fd, "SEEK_SET(25)");
char buf5[6] = {0};
read(fd, buf5, 5); std::cout << "Read 5 at 25: \"" << buf5 << "\"\n";

// — 3. SEEK_CUR: move relative to current —————
std::cout << "\n— 3. SEEK_CUR —\n";
lseek(fd, 0, SEEK_SET); // rewind
lseek(fd, 5, SEEK_CUR); // skip 5
showPosition(fd, "SEEK_CUR(+5)");
read(fd, &c, 1); std::cout << "Read: '" << c << "'\n";

lseek(fd, -3, SEEK_CUR); // back 3 (from pos 7, now pos 4)
showPosition(fd, "SEEK_CUR(-3)");
```

```
read(fd, &c, 1); std::cout << "Read: '" << c << "'\n";

// Get current position without moving:
off_t curPos = lseek(fd, 0, SEEK_CUR);
std::cout << "Current position (SEEK_CUR 0): " << curPos << "\n";

// — 4. SEEK_END: seek relative to end —————
std::cout << "\n— 4. SEEK_END —\n";
lseek(fd, -5, SEEK_END);
showPosition(fd, "SEEK_END(-5) - 5 from end");
char last5[6] = {0};
read(fd, last5, 5);
std::cout << "Last 5 chars: \"" << last5 << "\"\n";

lseek(fd, 0, SEEK_END);
showPosition(fd, "SEEK_END(0) - at end");

// — 5. Sparse files - seeking past end of file —————
std::cout << "\n— 5. Sparse file (seek past EOF) —\n";
int sparseFd = open("sparse.dat", O_RDWR | O_CREAT | O_TRUNC, 0644);

// Seek 1MB past start without writing
lseek(sparseFd, 1024 * 1024, SEEK_SET);
write(sparseFd, "END", 3); // only 3 bytes written

off_t sparseSize = lseek(sparseFd, 0, SEEK_END);
std::cout << "Sparse file reported size: " << sparseSize << " bytes\n";
std::cout << "(But only 3 bytes of actual data!)\n";
// On most filesystems, the hole (0 to 1MB-1) takes no disk space.
close(sparseFd);
unlink("sparse.dat");
```

```
// — 6. Random access – read any record by index —————
std::cout << "\n— 6. Random access records —\n";
struct Record { int id; char data[16]; double value; };

int recFd = open("records.bin", O_RDWR | O_CREAT | O_TRUNC, 0644);
Record recs[] = {
    {1, "alpha", 1.1},
    {2, "beta", 2.2},
    {3, "gamma", 3.3},
    {4, "delta", 4.4},
    {5, "epsilon", 5.5}
};
write(recFd, recs, sizeof(recs));

// Read record 3 (index 2) directly:
int targetIdx = 2;
off_t recOffset = static_cast<off_t>(targetIdx) * sizeof(Record);
lseek(recFd, recOffset, SEEK_SET);

Record r;
read(recFd, &r, sizeof(Record));
std::cout << "Record at index " << targetIdx << ": "
    << "id=" << r.id << " data=\"" << r.data
    << "\" value=" << r.value << "\n";

// Update record 1 (index 0) in place:
lseek(recFd, 0, SEEK_SET);
read(recFd, &r, sizeof(Record));
r.value = 99.9;
lseek(recFd, 0, SEEK_SET);
write(recFd, &r, sizeof(Record));
std::cout << "Updated record 0 value to 99.9\n";
```

```
// Verify update:
lseek(recFd, 0, SEEK_SET);
read(recFd, &r, sizeof(Record));
std::cout << "Verified: record 0 value = " << r.value << "\n";

close(recFd);
unlink("records.bin");
close(fd);
unlink("seek_demo.txt");

return 0;
}
```

Part 5: stat, fstat, access — File Metadata

Theory

`stat()` retrieves information about a file (size, permissions, timestamps, type) **without opening it**. `fstat()` does the same but takes an already-open file descriptor.

STRUCT stat – what it contains:

```
struct stat {  
    dev_t    st_dev;    // Device ID containing the file  
    ino_t    st_ino;    // Inode number (unique ID on filesystem)  
    mode_t   st_mode;   // File type + permissions  
    nlink_t  st_nlink;  // Number of hard links  
    uid_t    st_uid;    // Owner user ID  
    gid_t    st_gid;    // Owner group ID  
    dev_t    st_rdev;   // Device ID (for device files)  
    off_t    st_size;   // Total size in bytes  
    blksize_t st_blksize; // Preferred I/O block size  
    blkcnt_t st_blocks; // Allocated 512B blocks  
    time_t   st_atime;  // Last access time  
    time_t   st_mtime;  // Last modification time  
    time_t   st_ctime;  // Last status change time  
};
```

FILE TYPE MACROS (applied to st_mode):

```
S_ISREG(m) → regular file  
S_ISDIR(m) → directory  
S_ISLNK(m) → symbolic link  
S_ISSOCK(m) → socket  
S_ISFIFO(m) → named pipe (FIFO)  
S_ISCHR(m) → character device  
S_ISBLK(m) → block device
```



```
// file: stat_demo.cpp

#include <fcntl.h>
#include <unistd.h>
#include <sys/stat.h>
#include <sys/types.h>
#include <iostream>
#include <cstring>
#include <ctime>

void printStat(const struct stat& st, const std::string& label) {
    std::cout << "\n[" << label << "]\n";
    std::cout << "  Size:      " << st.st_size << " bytes\n";
    std::cout << "  Inode:     " << st.st_ino << "\n";
    std::cout << "  Hard links: " << st.st_nlink << "\n";
    std::cout << "  Blocks:    " << st.st_blocks << " (512-byte)\n";

    // File type
    std::cout << "  Type:      ";
    if (S_ISREG (st.st_mode)) std::cout << "regular file\n";
    else if (S_ISDIR (st.st_mode)) std::cout << "directory\n";
    else if (S_ISLNK (st.st_mode)) std::cout << "symbolic link\n";
    else if (S_ISFIFO(st.st_mode)) std::cout << "FIFO/pipe\n";
    else if (S_ISSOCK(st.st_mode)) std::cout << "socket\n";
    else
        std::cout << "other\n";

    // Permissions
    mode_t m = st.st_mode;
    char perm[11];
    perm[0] = S_ISDIR(m) ? 'd' : (S_ISLNK(m) ? 'l' : '-');
    perm[1] = (m & S_IRUSR) ? 'r' : '-';
```

```

perm[2] = (m & S_IWUSR) ? 'w' : '-';
perm[3] = (m & S_IXUSR) ? 'x' : '-';
perm[4] = (m & S_IRGRP) ? 'r' : '-';
perm[5] = (m & S_IWGRP) ? 'w' : '-';
perm[6] = (m & S_IXGRP) ? 'x' : '-';
perm[7] = (m & S_IROTH) ? 'r' : '-';
perm[8] = (m & S_IWOTH) ? 'w' : '-';
perm[9] = (m & S_IXOTH) ? 'x' : '-';
perm[10] = '\0';
std::cout << "  Permissions: " << perm << "\n";

// Timestamps
char tbuf[64];
std::strftime(tbuf, sizeof(tbuf), "%Y-%m-%d %H:%M:%S",
              std::localtime(&st.st_mtime));
std::cout << "  Modified:    " << tbuf << "\n";
std::strftime(tbuf, sizeof(tbuf), "%Y-%m-%d %H:%M:%S",
              std::localtime(&st.st_atime));
std::cout << "  Accessed:    " << tbuf << "\n";
}

int main() {
    std::cout << "
    stat() / fstat() - Complete Demo
    " << "\n";
    std::cout << "
    stat() / fstat() - Complete Demo
    " << "\n";
    std::cout << "
    " << "\n";

    // Create test file
    int fd = open("stat_test.txt", O_RDWR | O_CREAT | O_TRUNC, 0644);
    const char* content = "stat() demonstration file.\nSecond line.\n";
    write(fd, content, strlen(content));

    // — 1. stat() by path —————

```

```

struct stat st;
if (stat("stat_test.txt", &st) == 0)
    printStat(st, "stat(\"stat_test.txt\")");
else
    perror("stat");

// — 2. fstat() on open fd —————
if (fstat(fd, &st) == 0)
    printStat(st, "fstat(fd)");
else
    perror("fstat");

// — 3. lstat() — does NOT follow symlinks —————
symlink("stat_test.txt", "symlink_to_test.txt");
struct stat lst;
if (lstat("symlink_to_test.txt", &lst) == 0) {
    std::cout << "\n[lstat on symlink]\n";
    std::cout << "  Is symlink: " << (S_ISLNK(lst.st_mode) ? "YES" : "NO") << "\n";
    std::cout << "  Symlink size: " << lst.st_size << " bytes (length of target path)\n";
}
if (stat("symlink_to_test.txt", &st) == 0) {
    std::cout << "[stat on symlink — follows it]\n";
    std::cout << "  Is regular file: " << (S_ISREG(st.st_mode) ? "YES" : "NO") << "\n";
    std::cout << "  Size: " << st.st_size << " bytes\n";
}

// — 4. access() — check permissions without opening —————
std::cout << "\n— access() checks —\n";
struct { const char* path; int mode; const char* desc; } checks[] = {
    {"stat_test.txt", F_OK, "exists"},
    {"stat_test.txt", R_OK, "readable"},
    {"stat_test.txt", W_OK, "writable"},

```

```

    {"stat_test.txt", X_OK, "executable"},
    {"/etc/shadow", R_OK, "/etc/shadow readable"},
    {"/tmp", W_OK, "/tmp writable"},
    {nullptr, 0, nullptr}
};

for (int i = 0; checks[i].path; i++) {
    int ret = access(checks[i].path, checks[i].mode);
    std::cout << " access(\"" << checks[i].path
                << "\", " << checks[i].desc << "): "
                << (ret == 0 ? "YES" : "NO")
                << (ret != 0 ? std::string(" ") + strerror(errno) + "]" : "")
                << "\n";
}

// — 5. chmod() — change permissions —————
std::cout << "\n— chmod() —\n";
chmod("stat_test.txt", 0600); // owner rw only
stat("stat_test.txt", &st);
std::cout << "After chmod(0600): permissions = ";
std::cout << ((st.st_mode & 0777) == 0600 ? "0600 ✓" : "unexpected") << "\n";

chmod("stat_test.txt", 0644); // restore

close(fd);
unlink("stat_test.txt");
unlink("symlink_to_test.txt");
return 0;
}

```

Part 6: truncate, ftruncate

Theory

`truncate()` and `ftruncate()` resize a file to an exact length without reading or writing data.

SIGNATURES:

```
int truncate(const char* path, off_t length);  
int ftruncate(int fd, off_t length);
```

If length < current size → excess bytes REMOVED (truncation)
If length > current size → file EXTENDED with null bytes '\0'

USE CASES:

- Clear a log file without deleting it:
`truncate("app.log", 0);`
- Pre-allocate space for a database file:
`ftruncate(fd, 1024 * 1024 * 100); // pre-alloc 100 MB`
- Trim a file to its actual content length:
`ftruncate(fd, actualBytesUsed);`

```
// file: truncate_demo.cpp

#include <fcntl.h>
#include <unistd.h>
#include <sys/stat.h>
#include <iostream>
#include <cstring>

off_t fileSize(const char* path) {
    struct stat st;
    stat(path, &st);
    return st.st_size;
}

int main() {
    std::cout << "truncate() / ftruncate() - Demo\n";
    std::cout << "truncate() / ftruncate() - Demo\n";
    std::cout << "\n\n";

    // Create a file with 100 bytes
    int fd = open("trunc.txt", O_RDWR | O_CREAT | O_TRUNC, 0644);
    char data[100];
    memset(data, 'X', sizeof(data));
    write(fd, data, sizeof(data));
    std::cout << "Initial size: " << fileSize("trunc.txt") << " bytes\n";

    // Truncate to 40 bytes (shrink)
    ftruncate(fd, 40);
    std::cout << "After ftruncate(40): " << fileSize("trunc.txt") << " bytes\n";

    // Extend to 200 bytes (zero-fill the extra 160)
```

```
ftruncate(fd, 200);
std::cout << "After ftruncate(200): " << fileSize("trunc.txt") << " bytes\n";

// Read back – the extra bytes should be null
lseek(fd, 50, SEEK_SET);
char buf[10];
read(fd, buf, sizeof(buf));
std::cout << "Bytes 50-59 (should be 0): ";
bool allZero = true;
for (int i = 0; i < 10; i++) if (buf[i] != '\0') allZero = false;
std::cout << (allZero ? "all null ✓" : "unexpected!") << "\n";

close(fd);

// truncate() by path (file need not be open)
truncate("trunc.txt", 0);
std::cout << "After truncate(0) by path: " << fileSize("trunc.txt") << " bytes\n";

// Pre-allocate 1 MB (useful for database files)
ftruncate(open("prealloc.dat", O_RDWR | O_CREAT | O_TRUNC, 0644),
          1024 * 1024);
std::cout << "Pre-allocated file size: "
          << fileSize("prealloc.dat") << " bytes\n";

unlink("trunc.txt");
unlink("prealloc.dat");
return 0;
}
```

Part 7: dup, dup2 — File Descriptor Duplication

Theory

`dup()` and `dup2()` create a **copy** of a file descriptor that refers to the **same open file description** (same file, same position, same flags). Both old and new descriptors are interchangeable.

`dup(fd):`

- Returns a new fd (lowest available) pointing to the same file.
- Old fd and new fd share offset, flags, and mode.

`dup2(oldfd, newfd):`

- Makes newfd a duplicate of oldfd.
- If newfd is already open, it is closed first (atomically).
- Critical use: redirect stdin/stdout/stderr.

COMMON USE: I/O REDIRECTION

Shell command: `./program > output.txt`

is implemented as:

1. `open("output.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644) → fd=5`
2. `dup2(5, STDOUT_FILENO) // fd 1 now points to output.txt`
3. `close(5) // fd 5 no longer needed`
4. `exec("./program") // program writes to "stdout" = output.txt`


```
// file: dup_demo.cpp

#include <fcntl.h>
#include <unistd.h>
#include <sys/stat.h>
#include <iostream>
#include <cstring>

int main() {
    std::cout << "┌───────────────────────────────────┐\n";
    std::cout << "│      dup() and dup2() – Complete Demo      │\n";
    std::cout << "└───────────────────────────────────┘\n\n";

    // — 1. dup(): create a copy —————
    std::cout << "— 1. dup() basic usage —\n";
    int fd1 = open("dup_test.txt", O_RDWR | O_CREAT | O_TRUNC, 0644);
    std::cout << "fd1 = " << fd1 << "\n";

    int fd2 = dup(fd1);
    std::cout << "fd2 = dup(fd1) = " << fd2 << "\n";

    // Both refer to the same file, same position
    write(fd1, "Hello ", 6);      // write through fd1
    write(fd2, "World!\n", 7);    // write through fd2

    // Both share the offset – fd2 continues where fd1 left off
    lseek(fd1, 0, SEEK_SET);
    char buf[32] = {0};
    read(fd1, buf, sizeof(buf) - 1);
    std::cout << "File contents: \"" << buf << "\"\n";
```

```
// Closing fd1 does NOT close the file – fd2 still valid
close(fd1);
std::cout << "fd1 closed. fd2 still valid.\n";
write(fd2, "Still open!\n", 12);
close(fd2);

// — 2. dup2(): redirect stdout to a file —————
std::cout << "\n— 2. dup2(): redirect stdout —\n";

// Save original stdout
int savedStdout = dup(STDOUT_FILENO);
std::cout << "Saved stdout as fd=" << savedStdout << "\n";

// Open redirect target
int outFile = open("stdout_redirect.txt",
                  O_WRONLY | O_CREAT | O_TRUNC, 0644);

// Redirect stdout (fd=1) to outFile
dup2(outFile, STDOUT_FILENO);
close(outFile); // outFile no longer needed (stdout IS the file now)

// Now everything written to stdout goes to the file:
std::cout << "This goes to stdout_redirect.txt!\n";
write(STDOUT_FILENO, "Direct write to fd=1 also goes to file\n", 39);

// Restore stdout
dup2(savedStdout, STDOUT_FILENO);
close(savedStdout);

std::cout << "stdout restored. You can see this in the terminal.\n";

// Verify the file was written:
```

```
int verifyFd = open("stdout_redirect.txt", O_RDONLY);
char content[256] = {0};
read(verifyFd, content, sizeof(content) - 1);
close(verifyFd);
std::cout << "Contents of stdout_redirect.txt:\n" << content;

// — 3. dup2(): redirect stderr —————
std::cout << "\n— 3. dup2(): merge stderr into stdout —\n";
// This is equivalent to shell: program 2>&1
int savedStderr = dup(STDERR_FILENO);
dup2(STDOUT_FILENO, STDERR_FILENO); // stderr → same destination as stdout

write(STDERR_FILENO, "This error goes to stdout!\n", 27);

dup2(savedStderr, STDERR_FILENO); // restore stderr
close(savedStderr);

// — 4. dup2(): pipe between processes (conceptual) —————
std::cout << "\n— 4. pipe() + dup2() pattern —\n";
int pipefd[2];
pipe(pipefd); // pipefd[0]=read end, pipefd[1]=write end
std::cout << "Pipe: read_end=" << pipefd[0]
          << " write_end=" << pipefd[1] << "\n";

// Write to write end
write(pipefd[1], "Data through pipe\n", 18);
close(pipefd[1]);

// Read from read end
char pipeBuf[64] = {0};
read(pipefd[0], pipeBuf, sizeof(pipeBuf));
close(pipefd[0]);
```

```
std::cout << "Read from pipe: \"" << pipeBuf << "\"\n";

// Cleanup
unlink("dup_test.txt");
unlink("stdout_redirect.txt");
return 0;
}
```

Part 8: fcntl — File Control

Theory

`fcntl()` is the Swiss-army knife of file descriptor operations. It can get and set file descriptor flags, file status flags, advisory locks, and more.

SIGNATURE:

```
#include <fcntl.h>
```

```
int fcntl(int fd, int cmd, ...);
```

KEY COMMANDS:

F_DUPFD Duplicate fd (like dup()), but to fd $\geq$ arg)

F_DUPFD_CLOEXEC Duplicate with O_CLOEXEC set

F_GETFL Get file status flags (O_RDONLY, O_APPEND, etc.)

F_SETFL Set file status flags (add O_APPEND, O_NONBLOCK, etc.)

F_GETFD Get file descriptor flags (FD_CLOEXEC)

F_SETFD Set file descriptor flags

F_GETLK Get file lock info

F_SETLK Set file lock (non-blocking)

F_SETLKW Set file lock (blocking – waits)

```
// file: fcntl_demo.cpp

#include <fcntl.h>
#include <unistd.h>
#include <sys/stat.h>
#include <iostream>
#include <cstring>

int main() {
    std::cout << "
    std::cout << "
    std::cout << "

    int fd = open("fcntl_test.txt", O_RDWR | O_CREAT | O_TRUNC, 0644);
    write(fd, "fcntl demo content\n", 19);

    // — 1. F_GETFL: get current file status flags —————
    std::cout << "— 1. F_GETFL: get status flags —\n";
    int flags = fcntl(fd, F_GETFL);
    std::cout << "Flags raw value: 0x" << std::hex << flags
                << std::dec << "\n";
    std::cout << "O_RDONLY:  " << ((flags & O_ACCMODE) == O_RDONLY ? "yes":"no") << "\n";
    std::cout << "O_WRONLY:  " << ((flags & O_ACCMODE) == O_WRONLY ? "yes":"no") << "\n";
    std::cout << "O_RDWR:    " << ((flags & O_ACCMODE) == O_RDWR ? "yes":"no") << "\n";
    std::cout << "O_APPEND:  " << ((flags & O_APPEND) ? "yes":"no") << "\n";
    std::cout << "O_NONBLOCK:" << ((flags & O_NONBLOCK) ? "yes":"no") << "\n";

    // — 2. F_SETFL: add O_APPEND to existing flags —————
    std::cout << "\n— 2. F_SETFL: add O_APPEND —\n";
    flags |= O_APPEND;
    fcntl(fd, F_SETFL, flags);
```

```
flags = fcntl(fd, F_GETFL); // re-read to confirm
std::cout << "After F_SETFL: O_APPEND = "
           << ((flags & O_APPEND) ? "YES ✓" : "no") << "\n";

// All subsequent writes go to end of file regardless of lseek
lseek(fd, 0, SEEK_SET); // try to seek to beginning
write(fd, "Appended!\n", 10); // goes to END anyway due to O_APPEND

// — 3. F_GETFD / F_SETFD: file descriptor flags —————
std::cout << "\n— 3. FD_CLOEXEC flag —\n";
int fdFlags = fcntl(fd, F_GETFD);
std::cout << "FD_CLOEXEC before: "
           << ((fdFlags & FD_CLOEXEC) ? "set" : "not set") << "\n";

fcntl(fd, F_SETFD, fdFlags | FD_CLOEXEC);
fdFlags = fcntl(fd, F_GETFD);
std::cout << "FD_CLOEXEC after: "
           << ((fdFlags & FD_CLOEXEC) ? "set ✓" : "not set") << "\n";
// Now fd will be closed automatically if exec() is called

// — 4. F_DUPFD: duplicate to lowest fd >= arg —————
std::cout << "\n— 4. F_DUPFD —\n";
int newFd = fcntl(fd, F_DUPFD, 10); // dup to fd >= 10
std::cout << "F_DUPFD(10): new fd = " << newFd << "\n";
close(newFd);

// — 5. Advisory file locking (F_SETLK / F_SETLKW) —————
std::cout << "\n— 5. Advisory file locking —\n";
struct flock lock;

// Set a write lock on first 10 bytes
```

```
lock.l_type = F_WRLCK; // write lock
lock.l_whence = SEEK_SET;
lock.l_start = 0;
lock.l_len = 10; // lock first 10 bytes (0 = whole file)
lock.l_pid = 0;

if (fcntl(fd, F_SETLK, &lock) == 0) {
    std::cout << "Write lock acquired on bytes 0-9\n";
} else {
    std::cout << "Lock failed: " << strerror(errno) << "\n";
}

// Query the lock we just set:
lock.l_type = F_RDLCK; // ask: would a read lock succeed?
lock.l_pid = 0;
fcntl(fd, F_GETLK, &lock);
if (lock.l_type == F_UNLCK)
    std::cout << "No conflicting lock found\n";
else
    std::cout << "Conflicting lock held by pid=" << lock.l_pid << "\n";

// Release the lock
lock.l_type = F_UNLCK;
lock.l_whence = SEEK_SET;
lock.l_start = 0;
lock.l_len = 10;
fcntl(fd, F_SETLK, &lock);
std::cout << "Lock released\n";

close(fd);
unlink("fcntl_test.txt");
```



```
    return 0;  
}
```

Part 9: Directory Operations

Theory

POSIX DIRECTORY FUNCTIONS:

DIR*	opendir(const char* path)	→ open a directory stream
struct dirent*	readdir(DIR* dirp)	→ read next entry
int	closedir(DIR* dirp)	→ close directory stream
void	rewinddir(DIR* dirp)	→ rewind to beginning
int	mkdir(const char* path, mode_t mode)	→ create directory
int	rmdir(const char* path)	→ remove empty directory
int	rename(const char* old, const char* new)	→ rename/move
int	unlink(const char* path)	→ delete a file
char*	getcwd(char* buf, size_t size)	→ get working directory
int	chdir(const char* path)	→ change working directory

```
// file: directory_demo.cpp

#include <fcntl.h>
#include <unistd.h>
#include <sys/stat.h>
#include <dirent.h>
#include <iostream>
#include <cstring>
#include <string>
#include <vector>
#include <algorithm>

// Recursive directory listing
void listDir(const std::string& path, int depth = 0) {
    DIR* dir = opendir(path.c_str());
    if (!dir) { perror("opendir: " + path.c_str()); return; }

    struct dirent* entry;
    std::vector<std::string> names;

    while ((entry = readdir(dir)) != nullptr) {
        std::string name = entry->d_name;
        if (name == "." || name == "..") continue;
        names.push_back(name);
    }
    closedir(dir);

    std::sort(names.begin(), names.end());

    for (const auto& name : names) {
        std::string fullPath = path + "/" + name;
```

```

    struct stat st;
    lstat(fullPath.c_str(), &st);

    std::string indent(depth * 2, ' ');
    std::cout << indent;

    if (S_ISDIR(st.st_mode)) {
        std::cout << "[DIR] " << name << "/\n";
        listDir(fullPath, depth + 1);
    } else if (S_ISLNK(st.st_mode)) {
        char target[256] = {0};
        readlink(fullPath.c_str(), target, sizeof(target));
        std::cout << "[LINK] " << name << " → " << target << "\n";
    } else {
        std::cout << "[FILE] " << name
                    << " (" << st.st_size << " bytes)\n";
    }
}

}

int main() {
    std::cout << "┌───────────────────────────────────┐\n";
    std::cout << "│   Directory Operations – Complete Demo   │\n";
    std::cout << "└───────────────────────────────────┘\n\n";

    // — 1. Create directory structure —————
    std::cout << "— 1. mkdir —\n";
    mkdir("testdir", 0755);
    mkdir("testdir/subdir1", 0755);
    mkdir("testdir/subdir2", 0755);
    mkdir("testdir/subdir1/deep", 0755);
    std::cout << "Created testdir/ with subdirs\n";
}

```

```
// Create some files
auto makeFile = [](const std::string& path, const std::string& content) {
    int fd = open(path.c_str(), O_WRONLY | O_CREAT | O_TRUNC, 0644);
    write(fd, content.c_str(), content.size());
    close(fd);
};

makeFile("testdir/readme.txt", "Project readme\n");
makeFile("testdir/main.cpp", "int main(){\n");
makeFile("testdir/subdir1/util.cpp", "// utilities\n");
makeFile("testdir/subdir1/deep/data.bin", "\x00\x01\x02");
makeFile("testdir/subdir2/config.txt", "key=value\n");
symlink("../readme.txt", "testdir/subdir2/readme_link");

// — 2. List directory recursively —————
std::cout << "\n— 2. Directory listing —\n";
listDir("testdir");

// — 3. opendir / readdir / closedir manually —————
std::cout << "\n— 3. Manual readdir —\n";
DIR* dir = opendir("testdir");
struct dirent* entry;
std::cout << "Direct children of testdir/:\n";
while ((entry = readdir(dir)) != nullptr) {
    if (entry->d_name[0] == '.') continue;
    std::cout << "  d_name=" << entry->d_name;
    // d_type (not always available on all filesystems):
    switch (entry->d_type) {
        case DT_REG: std::cout << " [regular]"; break;
        case DT_DIR: std::cout << " [directory]"; break;
        case DT_LNK: std::cout << " [symlink]"; break;
        default:     std::cout << " [other]"; break;
    }
}
```

```
    }
    std::cout << "\n";
}
closedir(dir);

// — 4. getcwd and chdir —————
std::cout << "\n— 4. getcwd / chdir —\n";
char cwd[1024];
getcwd(cwd, sizeof(cwd));
std::cout << "Current dir: " << cwd << "\n";

chdir("testdir");
getcwd(cwd, sizeof(cwd));
std::cout << "After chdir to testdir: " << cwd << "\n";

chdir(".."); // go back
getcwd(cwd, sizeof(cwd));
std::cout << "After chdir ..: " << cwd << "\n";

// — 5. rename() —————
std::cout << "\n— 5. rename —\n";
rename("testdir/readme.txt", "testdir/README.md");
std::cout << "Renamed readme.txt → README.md\n";

// — 6. Cleanup: remove directory tree —————
std::cout << "\n— 6. Cleanup —\n";
unlink("testdir/subdir2/readme_link");
unlink("testdir/subdir2/config.txt");
rmdir("testdir/subdir2");
unlink("testdir/subdir1/deep/data.bin");
rmdir("testdir/subdir1/deep");
unlink("testdir/subdir1/util.cpp");
```

```
    rmdir("testdir/subdir1");  
    unlink("testdir/main.cpp");  
    unlink("testdir/README.md");  
    rmdir("testdir");  
    std::cout << "Cleaned up testdir/\n";  
  
    return 0;  
}
```

Part 10: Memory-Mapped Files — mmap

Theory

`mmap()` maps a file (or anonymous memory) directly into the process's virtual address space. Instead of using `read()` / `write()`, you access the file like a regular array in memory.

MMAP CONCEPT:

Without mmap:	With mmap:
<code>open() → fd</code>	<code>open() → fd</code>
<code>read(fd, buf, N)</code>	<code>char* p = mmap(NULL, size, PROT_READ,</code>
<code>process buf</code>	<code>MAP_SHARED, fd, 0)</code>
<code>write(fd, buf, N)</code>	<code>process p[i] directly!</code>
	<code>munmap(p, size)</code>

The OS maps the file's pages into your address space.

Reads → page faults load data from disk lazily.

Writes → dirty pages flushed to disk by OS (or `msync()`).

ADVANTAGES:

- ✓ Zero-copy: no kernel→userspace copy on read
- ✓ Can be shared between processes (`MAP_SHARED`)
- ✓ OS handles caching and paging automatically
- ✓ Natural random access: just `p[offset]`
- ✓ Excellent for large files (only accessed pages loaded)

SIGNATURE:

```
void* mmap(void* addr, size_t length, int prot, int flags,  
           int fd, off_t offset);
```

`addr` → suggested address (`NULL` = let OS choose)

`length` → size of mapping in bytes

`prot` → `PROT_READ` | `PROT_WRITE` | `PROT_EXEC` | `PROT_NONE`

`flags` → `MAP_SHARED` (changes visible to others / written to file)

`MAP_PRIVATE` (copy-on-write – changes private to process)

`MAP_ANONYMOUS` (no file – used for IPC / custom allocators)

fd → file descriptor (-1 for MAP_ANONYMOUS)
offset → byte offset in file (must be page-aligned, usually 0)

Returns: pointer to mapped region, or MAP_FAILED on error

int munmap(void* addr, size_t length); → unmap the region
int msync(void* addr, size_t length, int flags); → flush to disk


```
// file: mmap_demo.cpp

#include <fcntl.h>
#include <unistd.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <iostream>
#include <cstring>
#include <cerrno>

int main() {
    std::cout << "┌───────────────────────────────────┐\n";
    std::cout << "│          mmap() / munmap() – Complete Demo          │\n";
    std::cout << "└───────────────────────────────────┘\n\n";

    // — 1. Read a file via mmap —————
    std::cout << "— 1. Read file via mmap —\n";
    // Create test file
    int fd = open("mmap_test.txt", O_RDWR | O_CREAT | O_TRUNC, 0644);
    const char* original = "Hello, mmap world! ABCDEFGHIJKLMNOPQRSTUVWXYZ\n";
    write(fd, original, strlen(original));

    struct stat st;
    fstat(fd, &st);
    size_t fileLen = static_cast<size_t>(st.st_size);

    // Map entire file read-only
    char* mapped = static_cast<char*>(
        mmap(nullptr, fileLen, PROT_READ, MAP_PRIVATE, fd, 0));

    if (mapped == MAP_FAILED) { perror("mmap"); return 1; }
```

```
std::cout << "File size: " << fileLen << " bytes\n";
std::cout << "Mapped at: " << static_cast<void*>(mapped) << "\n";
std::cout << "Contents:  \n";
for (size_t i = 0; i < fileLen - 1; i++) std::cout << mapped[i];
std::cout << "\n\n";

// Random access – just use array indexing!
std::cout << "Byte at index 7: '" << mapped[7] << "'\n";
std::cout << "Last byte: '" << mapped[fileLen - 1] << "'\n";

munmap(mapped, fileLen);
std::cout << "Unmapped.\n\n";

// — 2. Modify file via mmap (MAP_SHARED) —————
std::cout << "— 2. Write through mmap (MAP_SHARED) —\n";
mapped = static_cast<char*>(
    mmap(nullptr, fileLen, PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0));

if (mapped == MAP_FAILED) { perror("mmap write"); return 1; }

// Modify in place – no write() call needed!
memcpy(mapped, "HELLO", 5);
std::cout << "Modified first 5 bytes to 'HELLO'\n";

// Force flush to disk (optional – OS may delay it)
msync(mapped, fileLen, MS_SYNC);
std::cout << "msync'd to disk.\n";

munmap(mapped, fileLen);

// Verify the file was actually changed:
```

```

lseek(fd, 0, SEEK_SET);
char verify[8] = {0};
read(fd, verify, 5);
std::cout << "Verify file first 5 bytes: \"" << verify << "\"\n\n";
close(fd);

// — 3. Anonymous mapping (no file – shared memory) —————
std::cout << "— 3. Anonymous mmap (shared memory between fork) —\n";
long pageSize = sysconf(_SC_PAGESIZE);
std::cout << "System page size: " << pageSize << " bytes\n";

// MAP_ANONYMOUS | MAP_SHARED → shared between parent and child
int* sharedMem = static_cast<int*>(
    mmap(nullptr, sizeof(int),
        PROT_READ | PROT_WRITE,
        MAP_ANONYMOUS | MAP_SHARED,
        -1, 0));

if (sharedMem == MAP_FAILED) { perror("anon mmap"); return 1; }
*sharedMem = 0; // initialise counter

pid_t pid = fork();
if (pid == 0) {
    // Child process
    for (int i = 0; i < 1000; i++) (*sharedMem)++;
    munmap(sharedMem, sizeof(int));
    _exit(0);
} else {
    // Parent process
    waitpid(pid, nullptr, 0);
    std::cout << "Shared counter after child incremented 1000x: "
        << *sharedMem << "\n";
}

```

```
    munmap(sharedMem, sizeof(int));
}

// — 4. mmap for large file processing (efficient) —————
std::cout << "\n— 4. Count newlines in file via mmap —\n";
// Create a larger file
int bigFd = open("big_test.txt", O_RDWR | O_CREAT | O_TRUNC, 0644);
for (int i = 0; i < 1000; i++) {
    char line[64];
    int len = snprintf(line, sizeof(line), "Line %04d: data\n", i);
    write(bigFd, line, static_cast<size_t>(len));
}
fstat(bigFd, &st);
size_t bigLen = static_cast<size_t>(st.st_size);

char* bigMap = static_cast<char*>(
    mmap(nullptr, bigLen, PROT_READ, MAP_PRIVATE, bigFd, 0));
close(bigFd);

int newlines = 0;
for (size_t i = 0; i < bigLen; i++)
    if (bigMap[i] == '\n') newlines++;

munmap(bigMap, bigLen);
std::cout << "File size: " << bigLen
    << " bytes | Newline count: " << newlines << "\n";

// Cleanup
unlink("mmap_test.txt");
unlink("big_test.txt");
```

```
return 0;  
}
```

Part 11: File Locking

Theory

File locking coordinates concurrent access to files between processes.

TWO LOCKING MECHANISMS:

`flock(fd, operation):`

- BSD-style locking – locks the WHOLE FILE
- `LOCK_SH` → shared (read) lock (multiple readers OK)
- `LOCK_EX` → exclusive (write) lock (only one at a time)
- `LOCK_UN` → unlock
- `LOCK_NB` → non-blocking (return error instead of waiting)
- Simple but limited – whole-file only

`fcntl(fd, F_SETLK/F_SETLKW, &flock):`

- POSIX advisory locking – locks BYTE RANGES
- More flexible: lock any range of bytes within a file
- `F_RDLCK` → shared read lock
- `F_WRLCK` → exclusive write lock
- `F_UNLCK` → unlock
- `l_whence + l_start + l_len` define the byte range

IMPORTANT: These are ADVISORY locks – they only work if all processes agree to use them. They do NOT prevent a process that ignores locks from reading/writing the file.

```
// file: locking_demo.cpp

#include <fcntl.h>
#include <unistd.h>
#include <sys/file.h>
#include <sys/stat.h>
#include <sys/wait.h>
#include <iostream>
#include <cstring>

// Helper: try to acquire flock, report result
bool tryFlock(int fd, int type, const std::string& label) {
    int nonBlock = (type & LOCK_NB);
    int ret = flock(fd, type);
    std::cout << "[" << getpid() << "] " << label << ": ";
    if (ret == 0) {
        std::cout << "acquired ✓\n";
        return true;
    } else {
        std::cout << "FAILED - " << strerror(errno) << "\n";
        return false;
    }
}

int main() {
    std::cout << "
    std::cout << "
    std::cout << "
    File Locking – Complete Demo
    \n\n";

    // Create shared lock file
    int fd = open("lockfile.dat", O_RDWR | O_CREAT | O_TRUNC, 0644);
```

```
const char* data = "Shared file content\n";
write(fd, data, strlen(data));

// — 1. flock() – whole-file locking —————
std::cout << "— 1. flock() whole-file locking —\n";

// Acquire exclusive lock
tryFlock(fd, LOCK_EX, "LOCK_EX (exclusive)");

// Try non-blocking lock in child – should fail while parent holds it
pid_t pid = fork();
if (pid == 0) {
    // Child: open file separately and try to lock
    int childFd = open("lockfile.dat", O_RDWR);
    tryFlock(childFd, LOCK_EX | LOCK_NB, "child LOCK_EX | LOCK_NB");
    close(childFd);
    _exit(0);
}
waitpid(pid, nullptr, 0);

flock(fd, LOCK_UN);
std::cout << "[" << getpid() << "] Released exclusive lock\n\n";

// Multiple shared locks can coexist:
std::cout << "— Multiple shared (LOCK_SH) locks —\n";
int fd2 = open("lockfile.dat", O_RDONLY);
int fd3 = open("lockfile.dat", O_RDONLY);

flock(fd2, LOCK_SH);
std::cout << "fd2 shared lock: acquired\n";
flock(fd3, LOCK_SH);
std::cout << "fd3 shared lock: acquired (simultaneously OK)\n";
```



```
// Both can read concurrently

flock(fd2, LOCK_UN);
flock(fd3, LOCK_UN);
close(fd2);
close(fd3);
std::cout << "Both shared locks released\n\n";

// — 2. fcntl() byte-range locking —————

std::cout << "— 2. fcntl() byte-range locking —\n";

// Lock first 10 bytes for writing
struct flock fl;
fl.l_type = F_WRLCK;
fl.l_whence = SEEK_SET;
fl.l_start = 0;
fl.l_len = 10;

if (fcntl(fd, F_SETLK, &fl) == 0) {
    std::cout << "Write lock on bytes 0-9 acquired\n";
}

// Lock bytes 20-29 for reading (shared)
fl.l_type = F_RDLCK;
fl.l_start = 20;
fl.l_len = 10;
if (fcntl(fd, F_SETLK, &fl) == 0) {
    std::cout << "Read lock on bytes 20-29 acquired\n";
}

// Query lock status on byte 5
fl.l_type = F_WRLCK;
```

```
f1.l_whence = SEEK_SET;
f1.l_start  = 5;
f1.l_len    = 1;
f1.l_pid    = 0;
fcntl(fd, F_GETLK, &f1);
if (f1.l_type == F_UNLCK)
    std::cout << "Byte 5: no conflicting lock\n";
else
    std::cout << "Byte 5: locked by pid=" << f1.l_pid << "\n";

// Release all locks by closing or explicitly unlocking
f1.l_type = F_UNLCK;
f1.l_start = 0;
f1.l_len = 0; // 0 = unlock entire file
fcntl(fd, F_SETLK, &f1);
std::cout << "All fcntl locks released\n\n";

// — 3. Lock file pattern (preventing duplicate processes) —————
std::cout << "— 3. Lock file pattern (single-instance daemon) —\n";

int lockFd = open("daemon.lock", O_RDWR | O_CREAT, 0644);

// Try to acquire exclusive lock (non-blocking)
if (flock(lockFd, LOCK_EX | LOCK_NB) == 0) {
    // Write PID to lock file
    char pidStr[32];
    ftruncate(lockFd, 0);
    lseek(lockFd, 0, SEEK_SET);
    int len = snprintf(pidStr, sizeof(pidStr), "%d\n", getpid());
    write(lockFd, pidStr, static_cast<size_t>(len));

    std::cout << "Lock file acquired. PID " << getpid()
```

```
        << " is the only instance.\n";
std::cout << "Lock released on close.\n";
// Lock is released when lockFd is closed (process exit or close)
close(lockFd);
} else {
    // Read existing PID from lock file
    char pidBuf[32] = {0};
    read(lockFd, pidBuf, sizeof(pidBuf));
    std::cout << "Another instance is running! PID from lock file: "
        << pidBuf;
    close(lockFd);
}

close(fd);
unlink("lockfile.dat");
unlink("daemon.lock");
return 0;
}
```

Part 12: Complete Reference and Comparison Table

System Call Quick Reference

LOW-LEVEL FILE I/O – COMPLETE QUICK REFERENCE	
OPEN / CLOSE	
<code>open(path, flags)</code>	Open/create file → fd. Flags: <code>O_RDONLY</code> , <code>O_WRONLY</code> , <code>O_RDWR</code> , <code>O_CREAT</code> , <code>O_TRUNC</code> , <code>O_APPEND</code> , <code>O_EXCL</code> , <code>O_NONBLOCK</code> , <code>O_CLOEXEC</code>
<code>open(path,flags,mode)</code>	<code>O_CREAT</code> needs mode (e.g. <code>0644</code>)
<code>close(fd)</code>	Release fd. Always check return value!
READ / WRITE	
<code>read(fd,buf,n)</code>	Read up to n bytes. Returns bytes read, <code>0=EOF</code> , <code>-1=error</code> . May return < n bytes!
<code>write(fd,buf,n)</code>	Write up to n bytes. Returns bytes written. May return < n bytes! Always loop to write all.
SEEK / POSITION	
<code>lseek(fd,off,SEEK_SET)</code>	Set position to off from beginning
<code>lseek(fd,off,SEEK_CUR)</code>	Move off bytes from current position
<code>lseek(fd,off,SEEK_END)</code>	Set position to off from end
<code>lseek(fd,0,SEEK_CUR)</code>	Get current position (like <code>ftell</code>)

lseek(fd,0,SEEK_END)	Get file size
METADATA	
stat(path, &st)	File metadata by path (follows symlinks)
fstat(fd, &st)	File metadata by fd
lstat(path, &st)	Metadata (does NOT follow symlinks)
access(path, mode)	Check F_OK/R_OK/W_OK/X_OK
chmod(path, mode)	Change permissions
chown(path,uid,gid)	Change owner
RESIZE	
truncate(path, len)	Resize file by path
ftruncate(fd, len)	Resize file by fd
DUPLICATE	
dup(fd)	Duplicate to lowest available fd
dup2(oldfd, newfd)	Duplicate oldfd to specific newfd
FILE CONTROL	
fcntl(fd,F_GETFL)	Get file status flags
fcntl(fd,F_SETFL,f)	Set file status flags (add O_APPEND etc.)
fcntl(fd,F_SETLK,&l)	Set byte-range lock (non-blocking)
fcntl(fd,F_SETLKW,l)	Set byte-range lock (blocking)
DIRECTORY	
opendir(path)	Open directory → DIR*
readdir(dir)	Read next entry → struct dirent*

	closedir(dir)		Close directory stream	
	mkdir(path, mode)		Create directory	
	rmdir(path)		Remove empty directory	
	unlink(path)		Delete a file	
	rename(old, new)		Rename or move file	
	getcwd(buf, size)		Get current working directory	
	chdir(path)		Change working directory	
	MEMORY MAPPING			
	mmap(0,len,prot,		Map file into address space	
	flags,fd,off)		prot: PROT_READ WRITE flags: MAP_SHARED	
	munmap(ptr, len)		Unmap region	
	msync(ptr,len,flags)		Flush changes to disk (MS_SYNC/MS_ASYNC)	
	LOCKING			
	flock(fd, op)		Whole-file lock: LOCK_SH/EX/UN [LOCK_NB]	

High-Level vs Low-Level Comparison

Operation	High-Level (C/C++)	Low-Level (POSIX)
Open file	fopen()	open()
Close file	fclose()	close()
Read bytes	fread()	read()
Write bytes	fwrite()	write()
Seek position	fseek()	lseek()
Get position	ftell()	lseek(fd,0,SEEK_CUR)
Rewind	rewind()	lseek(fd,0,SEEK_SET)
Check EOF	feof()	read() returns 0
Print text	fprintf()	write() with snprintf()
Scan text	fscanf()	read() + manual parsing
File size	fseek+ftell	stat() or lseek(SEEK_END)
Handle type	FILE*	int (fd)
Buffered?	YES (user-space)	NO (kernel buffers only)
Error check	ferror(), errno	errno always
Conversion	fileno(fp)→fd	fdopen(fd,"r")→FILE*

Common Mistakes

OPEN:

- ✗ Not checking `fd == -1` after `open()`
`int fd = open("file.txt", O_RDONLY);`
`read(fd, buf, 100);` // CRASH if `fd == -1` (reading `fd -1` is UB)
Fix: always check if `(fd == -1) { perror("open"); return 1; }`
- ✗ Using `O_CREAT` without mode argument
`open("new.txt", O_CREAT | O_WRONLY);` // mode is undefined!
Fix: `open("new.txt", O_CREAT | O_WRONLY, 0644);`

READ/WRITE:

- ✗ Assuming `read()` returns exactly count bytes
`read(fd, buf, 1000);`
process `buf` as if it has 1000 bytes // WRONG – may have fewer!
Fix: loop until all bytes read or EOF.
- ✗ Not null-terminating char buffers after `read()`
`char buf[100];`
`int n = read(fd, buf, 100);`
`printf("%s\n", buf);` // `buf` may not be null-terminated!
Fix: `buf[n] = '\0';` after `read()`.
- ✗ Ignoring `write()` short returns (disk full, signal)
`write(fd, data, len);` // may write fewer bytes!
Fix: use `writeAll()` loop that retries until `len` bytes written.

LSEEK:

- ✗ Seeking a non-seekable `fd` (pipe, socket, terminal)

`lseek` returns `-1` with `errno=ESPIPE`

Fix: check return value; use `stat()` to verify file type first.

CLOSE:

✗ Double-close: closing `fd` twice

```
close(fd); close(fd); // second close may close someone else's fd!
```

Fix: set `fd = -1` after `close()`.

✗ Ignoring `close()` return value on network file systems

`close()` can return `EIO` on NFS if a write was buffered and fails.

Fix: `if (close(fd) == -1) perror("close – data may be lost!");`

MMAP:

✗ Accessing `mmap` beyond the mapped region → `SIGBUS`

```
char* p = mmap(NULL, 10, PROT_READ, MAP_SHARED, fd, 0);
```

```
char c = p[15]; // SIGBUS or SIGSEGV!
```

Fix: always use exactly the length you mapped.

✗ Using `mmap` on a file that may be resized by another process

If file shrinks, accessing mapped pages past new end → `SIGBUS`.



Additional Resources

Books

- **"The Linux Programming Interface" by Michael Kerrisk** — The definitive 1550-page guide to POSIX system calls. Every function in this chapter is covered exhaustively with edge cases, rationale, and examples.
- **"Advanced Unix Programming" by Richard Stevens (APUE)** — The classic. Chapters 3-5 cover file I/O system calls with exceptional clarity.

- **"Computer Systems: A Programmer's Perspective" by Bryant & O'Hallaron** — Chapter 10 covers system-level I/O perfectly for understanding what's happening underneath.

Online

- man7.org/linux/man-pages — Full Linux manual pages for every system call. `man 2 open`, `man 2 read`, `man 2 mmap` etc.
- pubs.opengroup.org/onlinepubs/9699919799 — The POSIX standard. Authoritative definition of every function.

Commands

```
man 2 open      # open() manual
man 2 read      # read() manual
man 2 lseek     # lseek() manual
man 2 stat      # stat() manual
man 2 mmap      # mmap() manual
man 2 fcntl     # fcntl() manual
strace ./myprogram # trace every system call made by a program
ltrace ./myprogram # trace library calls
```

 **Pro Tip:** The most important habit when writing low-level file code is **checking every return value**. `open()` can fail. `read()` can return fewer bytes than requested. `write()` can return fewer bytes than requested. `close()` can fail. None of these are hypothetical — they all happen in production. Every unchecked return value is a latent bug. A single `if (n == -1) { perror("read"); }` pattern repeated consistently will save you hours of debugging.

End: Low-Level File Operations — POSIX System Calls

Complete guide: open · close · read · write · lseek · stat · dup · fcntl · mmap · locking

Guide Version 1.0 — POSIX / Linux / macOS